

PRD: BilletVision — Automated Billet Inspection & Traceability

Event: Code Utsava X.0 (Turing Club of Programmers) | **Problem Statement:** 1 | **Duration:** 28 hours **Version:** 1.0 | **Date:** 3 Oct 2026

1. One-line pitch

A non-contact computer-vision system that watches billets pass on a conveyor, measures them to within $\pm 1\%$, reads their heat/batch markings, flags out-of-tolerance pieces instantly, and writes an audit-ready Excel/CSV log, with no human near the hot line.

2. Problem recap (from PS 1)

Manual billet inspection is slow, error-prone, dangerous near hot steel, and creates bad records. We need: live capture, dimension + defect measurement, OCR/barcode reading, automated Excel logging, instant tolerance alerts, and an operator dashboard.

3. Goals, non-goals, success metrics

Goals (must hit by hour 28)

#	Goal	Target (from PS)
G1	Dimensional accuracy after calibration	$\leq \pm 1\%$ error vs. ground truth
G2	Speed	≥ 15 FPS live, or < 2 s inspection latency per billet
G3	OCR / ID reading	$\geq 90\%$ on test set of stamped/painted/printed IDs
G4	Logging	Append-mode Excel/CSV, no corruption, no file-lock crash during continuous run
G5	Alerts	Out-of-tolerance $\rightarrow$ visual + audio alert within 1 s of decision
G6	Robustness	Works under varied lighting, dust/noise, mild scale
G7	Dashboard	Live feed, bounding boxes, measurements, alert panel, log table

Non-goals (say these out loud in the pitch, since judges respect honesty)

- No claim of production deployment on a real hot mill. We prototype on video / scaled props.
- No custom-trained detector on thousands of real billet images (no public dataset to rely on).
- No PLC/MES integration (we show an export + REST hook as the integration path).

4. Users & scenarios

- **Line operator:** watches dashboard, hears alert, diverts the billet.
- **QA engineer:** reviews pass/fail history, filters by heat number, exports audit logs.
- **Plant manager:** looks at yield / rejection trends.

Primary scenario: billet enters the field of view $\rightarrow$ trigger $\rightarrow$ N frames captured $\rightarrow$ measured, ID read, verdict computed $\rightarrow$ row appended to log $\rightarrow$ dashboard updates $\rightarrow$ alert if FAIL.

5. Scope decisions and honest assumptions (decide these in hour 0-1)

Question	Decision	Why
Input source	Support 3 sources behind one interface: (a) webcam, (b) video file, (c) image folder	Demo never depends on one thing working
Test	Phone-recorded video of scaled billet props (wooden/aluminium/steel bars, PVC or metal blocks)	Nobody has a mill nearby;

material	moving on a belt or hand-slid on a table, with stamped/printed ID labels, plus any hot-billet / steel-bar footage you can find for the robustness slide	props give you <i>real ground truth</i> via calipers
Dimension model	Square/rectangular and round billet modes (config switch)	PS says "width, height/diameter"
Length of real billets (6–12 m)	One camera can't see that. Real answer = line-scan camera or multi-camera stitching or length from belt-speed × time-in-view. We implement time-in-view × calibrated belt speed as a length mode + direct measurement for props within the FOV	Shows you understand the real constraint
Calibration	Reference object of known size in the scene (ArUco marker or checkerboard) → pixels-per-mm; homography for perspective correction	Core to the ±1% claim
Hot billets	Glow → bright-on-dark; use intensity thresholding mode + ND/exposure lock; note IR/thermal camera as production path	Keeps it realistic

6. System architecture

```

Camera / Video / Folder
|
[Capture thread] → frame queue (drop-oldest, keeps FPS up)
|
[Vision worker]
1. Preprocess (undistort, exposure normalise, CLAHE)
2. Segment billet (classical: threshold/Otsu/adaptive + morphology + contours; optional YOLOv8-seg)
3. Trigger / tracking (billet fully in ROI; centroid tracker → one ID per billet, no double count)
4. Measure (minAreaRect / ellipse fit → mm via calibration; ovality, diagonal diff, edge straightness)
5. Defect check (surface anomaly score, cross-section deviation, bend)
6. ID read (ROI crop → enhance → OCR + QR/barcode → regex validate → multi-frame vote)
7. Decision engine (tolerance config → PASS / FAIL / REWORK / REVIEW)
|
[Event bus]
├─> Logger (SQLite truth → CSV append → Excel writer thread, atomic)
├─> Alert manager (UI banner, sound, optional Telegram/webhook)
├─> WebSocket → Dashboard
|
[FastAPI backend] ↔ [Operator dashboard (browser)]

```

Tech stack (recommended, all free, all pip-installable)

- Python 3.11, OpenCV, NumPy, scikit-image
- OCR: PaddleOCR (best on stamped/odd fonts) with EasyOCR fallback, Tesseract as last resort; `pyzbar` / `cv2.QRCodeDetector` for codes
- Optional DL: Ultralytics YOLOv8n-seg (only if time permits; classical CV is the default and is faster and more explainable)
- Backend: FastAPI + Uvicorn, WebSocket for live events, MJPEG endpoint for the video feed
- Storage: SQLite (source of truth) → `pandas` + `openpyxl` / CSV export
- Frontend: single-page HTML + vanilla JS (or Streamlit if the team is weak at frontend; see §13)
- Tests: pytest

7. Functional requirements (what we build)

7.1 Capture & vision engine (Deliverable 1)

- FR-1 Pluggable source (webcam index / video path / folder) with FPS counter.
- FR-2 Bounded queue; if processing lags, drop frames, never grow latency.
- FR-3 Calibration wizard: detect ArUco/checkerboard → compute mm/px → save `calibration.json` ; re-calibrate from the UI.
- FR-4 Segmentation robust to lighting: adaptive threshold + background subtraction fallback + auto exposure/gamma normalisation.
- FR-5 Dimension outputs: length, width, height/diameter (mm), plus derived: ovality, diagonal difference (rhomboidity), straightness/camber.
- FR-6 Defect flags: cross-sectional variation (profile width sampled along the length, flag if std/max-min exceeds a limit), edge irregularity, surface anomaly (classical: texture/blob anomaly in the billet mask; optional autoencoder).
- FR-7 Tracking so each physical billet yields exactly one record, using the median of N good frames, not one noisy frame.

7.2 OCR & detail extraction (Deliverable 2)

- FR-8 ID-region localisation (text detector or fixed ROI per config).
- FR-9 Enhancement pipeline: grayscale → CLAHE → denoise → sharpen → deskew → upscale; try multiple variants and pick

the best by confidence.

- FR-10 Barcode/QR decode in parallel; QR wins if present.
- FR-11 Format validation: configurable regex (e.g. `^[A-Z]\d{5,7}$` for heat no.) and charset correction (O↔0, l↔1, S↔5, B↔8).
- FR-12 Multi-frame voting across the billet's track → single confident ID; if confidence < threshold, status = REVIEW and the crop is saved for the operator to confirm.

7.3 Data logging (Deliverable 3)

Columns: `timestamp, billet_seq, billet_id (heat), batch_id, length_mm, width_mm, height_mm, diameter_mm, ovality, diag_diff_mm, defects, ocr_confidence, status (PASS/FAIL/REWORK/REVIEW), fail_reasons, image_path, processing_ms`.

- FR-13 **Write path:** event → SQLite (instant, durable) → in-memory queue → single writer thread appends to CSV (always works) and to XLSX.
- FR-14 **No corruption / no lock crash:** XLSX is written to a temp file then atomically replaced; if Excel has the file open (PermissionError) → fall back to `log_YYYYMMDD_part2.xlsx` and merge later; CSV is never locked. Show "log health" on the dashboard.
- FR-15 Daily rotation, header auto-create, a duplicate-ID warning, and an export button (CSV / XLSX / JSON).

7.4 Operator dashboard & alerts (Deliverable 4)

- FR-16 Live video with bounding boxes + overlaid mm values + ID text.
- FR-17 Big status tile (PASS green / FAIL red flashing) + audio beep + alert history; optional Telegram/webhook push.
- FR-18 Tolerance editor in the UI (nominal ± tol for each dimension), applied live.
- FR-19 Log table (last 100, filter by status/heat), KPIs: total, pass rate, FPS, avg latency, OCR success rate.
- FR-20 Review queue: REVIEW items show the crop; operator types the right ID, saved back to the log.
- FR-21 Drill-down on any billet: frames, measurements, decision reasoning.

8. Differentiators (what makes us stand out, ranked by impact per hour)

Tier 1 — build these, they win

1. **Calibration-first, measurable accuracy.** A built-in *Accuracy Report*: measure 10+ props with calipers, run the system, and show a table of error % with mean and max, proving ≤1%. Almost nobody will do this.
2. **Explainable verdicts.** Every FAIL says why ("width 131.8 mm > 130 ± 1.0") and shows the annotated frame.
3. **Never lose a record.** The SQLite → CSV → XLSX atomic pipeline, with a live kill-Excel-while-running demo.
4. **Multi-frame voting for OCR + REVIEW queue.** The system knows when it is unsure and asks a human instead of logging garbage (directly addresses "corrupted records").
5. **Tracking = one record per billet**, with no duplicates and none missed.

Tier 2 — if time permits

6. Robustness switchboard: a live demo toggle simulating low light / glare / dust / motion blur (augmentation on the feed) showing the accuracy hold.
7. Hot-billet mode (glow segmentation) with a short clip.
8. Trend charts: dimension drift over time (SPC control chart with UCL/LCL → predictive "line drifting out of tolerance" warning before failures occur). This is a strong QA story.
9. Audio/voice alert + Telegram message with the annotated image.
10. Config-driven billet profiles (e.g. 130×130 mm, Ø150 mm) selectable in the UI.

Tier 3 — pitch-only (slide, not code)

- Thermal/IR camera and line-scan camera for production, PLC/OPC-UA/MES integration, edge deployment on Jetson, auto-diverter actuator signal.

9. 28-hour execution plan

Assume 4 people: **A** = Vision/measurement, **B** = OCR, **C** = Backend/logging/alerts, **D** = Dashboard + demo data + docs/pitch. (Scale down roles for 2–3 people: merge B into A, D into C.)

Hours	Milestone	A (Vision)	B (OCR)	C (Backend)	D (UI/Data/Pitch)
0-1	Kickoff	Read PS again together, lock scope (\$5), repo + env, assign roles			

1-3	Data + skeleton	Capture/source abstraction, queue, FPS counter	Collect 40+ ID images (stamped, painted, printed, blurry)	FastAPI skeleton, SQLite schema, config.yaml	Record test video + props, measure ground truth with calipers (very important), wireframe UI
3-7	Core v1	Segmentation + minAreaRect, calibration (ArUco), first mm outputs	OCR baseline (PaddleOCR/EasyOCR), accuracy on test set	Logger: SQLite→CSV, decision engine from tolerances	Dashboard v0: live feed + status tile via WebSocket
7-8	Checkpoint 1: end-to-end thin slice (video → measure → log → dashboard). Freeze interfaces.				
8-14	Depth	Tracking, multi-frame median, ovality/diagonal/camber, cross-section profile, defects	Preprocessing variants, regex+charset fix, QR/barcode, voting, REVIEW flow	XLSX atomic writer + lock fallback, alerts (sound/Telegram), export	Tolerance editor, log table, review queue, KPIs
14-15	Checkpoint 2: all 7 PS "must" bullets demoable. Tag a git release.				
15-19	Accuracy + robustness	Accuracy report (error % table), tune thresholds, lighting robustness, hot mode	OCR to ≥90% on test set; log failure cases	Latency profiling, 15+ FPS tuning, long-run soak test (no crash for 30+ min)	Drill-down, SPC chart, robustness toggle
19-22	Tests + hardening	Edge cases (partial billet, two billets, empty belt)	Edge cases (glare, rotated text)	pytest: decision engine, logger, concurrency	Polish UI, error states
22-25	Docs + pitch	Final accuracy numbers	Final OCR numbers	README, setup, architecture diagram	Slides, demo script, backup demo video
25-27	Dress rehearsal x3	Everyone: full 5-min pitch + Q&A drill, kill-switch drills (cable out, Excel open, camera lost)			
27-28	Freeze. No new features. Final commit, tag, backup laptop, backup video, USB copy.				

Rules: feature freeze at hour 22 for new features, hour 27 for everything. Merge to `main` only if the demo still runs. Commit at least hourly. Never start a Tier 2 item before Checkpoint 2.

10. Evaluation plan (numbers you will show on a slide)

Metric	How measured	Target
Dimension error	$N \geq 10$ props × 3 runs, calipers as ground truth, `	measured – true
Repeatability	Same prop 20 times, report std dev (mm)	< 0.3 mm
OCR accuracy	40+ ID images, exact-match rate (+ char-level accuracy)	≥ 90% exact
Throughput	FPS on a laptop CPU (state hardware)	≥ 15 FPS
Per-billet latency	Trigger → log row written	< 2 s
Tolerance decisions	Confusion matrix on a set with known good/bad props	0 missed FAILs on the test set
Log integrity	30-min soak + Excel opened/closed mid-run	0 lost / corrupted rows
Robustness	Accuracy under brightness ±40%, blur, noise, glare	Report the drop honestly

Report **misses and limitations** too. Judges trust teams that show failure cases.

11. Presentation & demo

11.1 Pitch structure (5-7 min)

1. **Hook (30 s):** the human standing next to a 1,100 °C billet with a tape measure and a clipboard. Cost: injuries, wrong records, scrap.
2. **Solution in one sentence + architecture diagram (45 s).**
3. **Live demo (2.5 min), see script below.**
4. **Numbers slide (45 s):** accuracy table, FPS, OCR %, latency.
5. **What makes us different (45 s):** calibration-proven accuracy, explainable decisions, no lost records, human-in-the-loop for uncertain IDs.
6. **Limits & roadmap (30 s):** thermal/line-scan cameras, PLC integration, diverter actuation, drift prediction.
7. **Close (15 s):** impact — safety, 100% traceability, throughput, less scrap.

11.2 Demo script (rehearse it to the second)

1. Open dashboard: show the live feed + calibration status ("1 px = 0.21 mm").
2. Run a good billet → PASS, ID read, row appears in Excel *live* (have Excel open next to it).
3. Run an oversized billet → red flash + beep + "width 131.8 > 130±1.0" → show the Telegram message if built.
4. Run a billet with a blurry ID → goes to REVIEW → operator fixes → log updated.
5. Toggle low-light / noise → still works; show the accuracy delta.
6. **Closing flourish:** show the Accuracy Report table + the SPC drift chart.
7. Keep a **pre-recorded backup video** of the same flow ready.

11.3 Presentation tips

- One backup per risk: laptop, video, offline wheels ([pip download](#)), mobile hotspot, spare camera.
- Lead with the problem and the impact, not the tech.
- Give each team member a part; everyone must be able to answer anything on the system.

12. Likely judge questions (and strong answers)

Accuracy & measurement

- *How do you get ±1%?* → Calibration object + homography to remove perspective; subpixel contour fitting; median across frames; validated against caliper ground truth (show table).
- *What about lens distortion and camera tilt?* → Camera calibration (undistort) and a plane homography; we recommend mounting orthogonal to the belt; tilt is corrected by the reference marker.
- *How do you measure a 12 m billet?* → Not with one frame: line-scan or multi-camera in production; our prototype uses belt-speed × time-in-view for length and direct measurement for FOV-fit pieces. We state this limitation up front.
- *Does thermal expansion matter?* → Yes, hot billets are ~1-2% longer in reality; production systems apply a temperature-based correction using an IR pyrometer. We expose this as a configurable factor.
- *What if the billet moves/vibrates?* → Short exposure, global-shutter or fast shutter, multi-frame median, and the tracker rejects blurred frames via a sharpness score.

OCR

- *How do you read stamped marks on rough steel?* → Contrast enhancement (CLAHE), directional lighting recommendation, multiple preprocessing variants, charset correction, format regex, multi-frame voting; uncertain reads go to human review instead of being logged blindly.
- *Where's your 90%?* → Show the test set size, exact-match vs char-level, and the failure cases.
- *Why not train a model?* → No public dataset of stamped billets; with limited time we use pretrained OCR + preprocessing + validation, and we've designed the data capture so the review queue builds a fine-tuning dataset.

Engineering

- *How do you avoid Excel corruption/lock?* → SQLite is the source of truth; single writer thread; atomic temp-file replace; CSV fallback; rotating part files if Excel holds the lock.
- *Classical CV or deep learning, and why?* → Classical for geometry because it's deterministic, fast, explainable, and easy to calibrate; DL (optional) for defects/text where learning is needed. Hybrid by design.
- *Latency / FPS?* → Show profiling numbers per stage; mention a frame-drop queue; GPU/Jetson as the scaling path.
- *How do you avoid double-logging the same billet?* → Centroid tracker with entry/exit line, one record per track ID.
- *What happens if the camera disconnects?* → Watchdog, reconnect, dashboard "camera lost" alert, log entry of the outage.

Business / impact

- *How does this save money?* → Early reject cuts downstream scrap, removes manual measurement time, avoids safety

incidents, gives complete audit trails (compliance).

- *How would you deploy it in a mill?* → IP-rated enclosure with air purge, IR-pass filter, strobe lighting, edge box, OPC-UA/PLC output to a diverter, MES connector.
- *What's the cost?* → Hardware approx. one industrial camera + edge computer + lighting; software is open source. (Give a rough figure only if you've checked prices.)
- *What would you do with another 2 weeks?* → Collect real data, fine-tune the detector and OCR, line-scan length measurement, thermal fusion, PLC integration, drift prediction.

13. Decisions to make early

- **Dashboard tech:** FastAPI + vanilla JS (smoother live video, more control) is the recommendation. Choose Streamlit only if nobody on the team is comfortable with JS. Streamlit video is laggy; in that case use `st.image` with a placeholder loop, or a native OpenCV window plus a Streamlit log page.
- **OCR engine:** time-box to 90 minutes comparing PaddleOCR vs EasyOCR on your own 40 images, then pick the winner on data, not opinion.
- **Detector:** start classical. Only move to YOLO-seg if segmentation fails on the test footage.

14. Prerequisite knowledge to pre-learn (ranked by priority)

Must know (study before the event)

1. **OpenCV fundamentals:** thresholding (Otsu/adaptive), morphology, `findContours`, `minAreaRect`, `fitEllipse`, `warpPerspective`, `undistort`, ArUco detection.
2. **Camera calibration:** pinhole model, `calibrateCamera` (checkerboard), homography, px→mm conversion, what causes measurement error (perspective, distortion, depth variation, blur, rolling shutter).
3. **Image enhancement for OCR:** CLAHE, bilateral/non-local-means denoise, unsharp mask, deskew, upscaling; why OCR fails on metal (specular glare, low contrast).
4. **OCR tools:** PaddleOCR/EasyOCR API, confidence scores, `pyzbar` for barcodes/QR.
5. **Concurrency in Python:** threads vs processes, `queue.Queue`, producer/consumer, why a single-writer pattern prevents file corruption, `asyncio` basics for FastAPI WebSockets.
6. **Excel/CSV I/O:** `openpyxl` append, file locking behaviour on Windows, atomic write (`os.replace`), SQLite basics.
7. **FastAPI:** routes, WebSocket, StreamingResponse for MJPEG.

Good to know

8. **Object tracking:** centroid tracker / IoU, entry-exit line counting.
9. **SPC (statistical process control):** control charts, UCL/LCL, Cp/Cpk basics, so you can speak the quality engineer's language.
10. **Steel billet domain basics:** continuous casting, billet vs bloom vs slab, typical sections (e.g. 100–165 mm square), heat number / batch / traceability, why dimensional defects matter (rhomboidity/diagonal difference, ovality, bending, surface cracks, scale), thermal expansion.
11. **Industrial vision hardware:** global vs rolling shutter, line-scan vs area-scan, telecentric lenses, IR-pass filters, strobe lighting, thermal cameras.
12. **YOLOv8-seg:** fine-tuning flow, in case you need it.

Pitch-level only

13. OPC-UA / PLC / MES at a one-paragraph level.

Tip: spend the pre-event hours *building* a 2-hour "measure a coin/ruler in a photo with ArUco calibration" mini-project and a "read text off a stamped metal photo" mini-project. It covers 70% of the risk.

15. Risks & fallbacks

Risk	Likelihood	Fallback
OCR below 90%	High	Controlled print/stamp style for props, ROI-fixed reading, QR/barcode as a first-class ID, REVIEW queue; report honest numbers
Segmentation fails under glare	Medium	Backlit/diffuse lighting for demo, background subtraction, switch to YOLO-seg

±1% not achieved	Medium	Larger reference marker, fix camera distance, per-axis correction factor, subpixel edges; report real numbers
Excel lock issue	Medium	CSV is primary, part files, atomic writes
Webcam/driver issues at venue	Medium	Video-file source is always ready
Dashboard lag	Medium	Lower the stream resolution/quality, JPEG quality 70, push only metadata over WebSocket
Integration chaos late	High	Interfaces frozen at hour 8; thin slice first
Demo failure	Medium	Pre-recorded backup, offline dependencies, rehearsed

16. Repo structure

```
billetvision/
├── CLAUDE.md
├── PRD.md
├── README.md
├── config/
├── src/billetvision/
│   ├── capture/
│   ├── vision/
│   ├── ocr/
│   ├── decision/
│   ├── logging/
│   ├── alerts/
│   ├── api/
│   └── pipeline.py
├── web/
├── data/
├── tests/
├── scripts/
└── docs/
```

```
# config.yaml, tolerances.yaml, billet_profiles.yaml, calibration.json
# sources.py (webcam/video/folder), queue
# preprocess.py, segment.py, measure.py, defects.py, track.py, calibrate.py
# enhance.py, reader.py, codes.py, validate.py, vote.py
# engine.py (tolerances → PASS/FAIL/REWORK/REVIEW + reasons)
# db.py, csv_writer.py, xlsx_writer.py
# manager.py, telegram.py
# main.py (FastAPI), ws.py, mjpeg.py
# wires everything together
# index.html, app.js, styles.css, alert.mp3
# raw/, ground_truth.csv, ocr_testset/, outputs/
# run_demo.py, accuracy_report.py, ocr_eval.py, soak_test.py
# architecture.png, accuracy_report.md, assumptions.md
```

17. Definition of done (checklist)

- ☐ Live/video feed processed ≥ 15 FPS with bounding boxes
- ☐ Calibration wizard works; `calibration.json` persisted
- ☐ Length/width/height/diameter + ovality/diagonal output in mm
- ☐ Accuracy report: max error ≤ 1% on ≥10 props
- ☐ Heat/batch ID via OCR + QR/barcode; ≥ 90% on test set
- ☐ PASS/FAIL/REWORK/REVIEW decision with reasons
- ☐ XLSX + CSV append logging, 30-min soak, Excel-open test passes
- ☐ Instant alert (visual + sound) on out-of-tolerance
- ☐ Dashboard: feed, tolerances, log, review queue, KPIs, export
- ☐ pytest suite green; README with one-command run
- ☐ Backup demo video recorded; pitch rehearsed 3×

Appendix A: Claude Code prompt sequence

Run these **in order**, one per session or sub-task, and end each with "run the tests and show me the result." Keep `CLAUDE.md` in the repo root so every prompt inherits the context.

- Scaffold:** "Read CLAUDE.md and PRD.md. Create the repo structure from PRD §16, `pyproject.toml` / `requirements.txt`, config YAMLS with sensible defaults, and a `run_demo.py` that prints 'ok'. No feature code yet."
- Capture:** "Implement `capture/sources.py` with a common `FrameSource` interface for webcam, video file, and image folder, a drop-oldest bounded queue, and an FPS counter. Add tests with a tiny generated video."
- Calibration:** "Implement `vision/calibrate.py`: detect an ArUco marker of known size, compute mm-per-pixel and a homography, save/load `calibration.json`. Add a CLI `scripts/calibrate.py` and unit tests using synthetic images."
- Segmentation + measurement:** "Implement `vision/segment.py` and `vision/measure.py` per CLAUDE.md: classical segmentation, `minAreaRect`, mm conversion, ovality and diagonal difference, round and square modes. Return a typed `Measurement` dataclass. Test against `data/ground_truth.csv`."
- Tracking + median:** "Implement `vision/track.py` (centroid tracker with an entry/exit line, one record per billet, median over the best-N frames by sharpness)."
- Decision engine:** "Implement `decision/engine.py` that loads `tolerances.yaml` and returns PASS/FAIL/REWORK/REVIEW"

with a list of human-readable reasons. Pure functions, 100% unit-tested."

7. **OCR:** "Implement `ocr/` : enhancement variants, PaddleOCR with EasyOCR fallback, QR/barcode decode, regex + charset correction, multi-frame voting, confidence. Write `scripts/ocr_eval.py` that prints exact-match and character accuracy on `data/ocr_testset/`."
8. **Logging:** "Implement `logging_` : SQLite as the source of truth, a single writer thread appending to CSV and to XLSX via temp-file + `os.replace` , with a `PermissionError` fallback to part files. Write a concurrency test and a 5-minute soak script."
9. **Pipeline + API:** "Wire everything in `pipeline.py` . Build the FastAPI app with an MJPEG `/video` , a WebSocket `/events` , REST for tolerances, log, export, and review-queue updates."
10. **Dashboard:** "Build `web/` single-page dashboard per PRD §7.4: live feed, big status tile, flashing alert with sound, tolerance editor, log table with filters, review queue, KPIs, drill-down."
11. **Alerts:** "Add `alerts/manager.py` (de-bounced) and an optional Telegram sender that sends the annotated image, enabled by env vars."
12. **Accuracy report:** "Write `scripts/accuracy_report.py` that runs the pipeline on the ground-truth set and writes `docs/accuracy_report.md` with a per-prop error table, mean/max error, and repeatability."
13. **Robustness:** "Add a runtime augmentation toggle (brightness, blur, noise, glare) in the pipeline and dashboard, plus a script that reports accuracy under each."
14. **Hardening:** "Add watchdog for camera loss, graceful error states, structured logging, and edge-case tests (partial billet, two billets, empty belt)."
15. **Docs:** "Write README (one-command setup and run), `docs/assumptions.md` , and a limitations section. Generate an architecture diagram."

Prompting tips: give Claude Code the failing test output, not a description; ask for small PRs; paste real ground-truth numbers; say "don't add features I didn't ask for"; after each step, commit.